TaskSAT

Scheduling and Verifying Task Networks with SMT

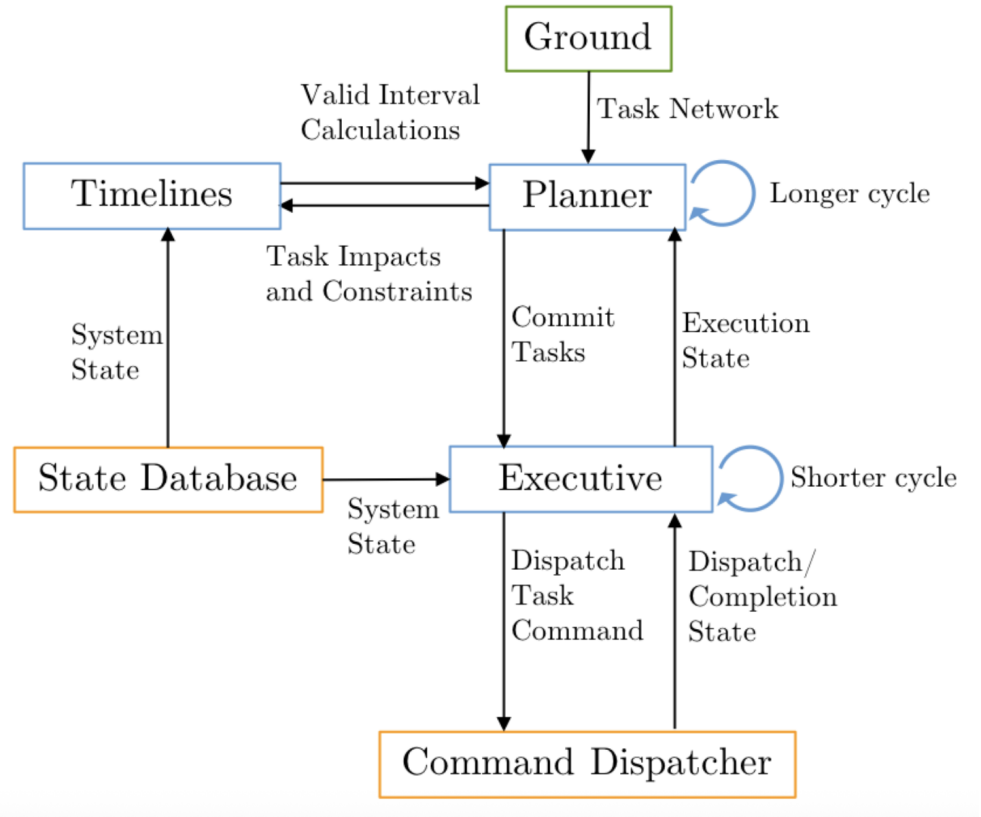
Klaus Havelund
Jet Propulsion Laboratory
California Institute of Technology



Jet Propulsion Laboratory
California Institute of Technology

JPL — 2026

The setting: MEXEC

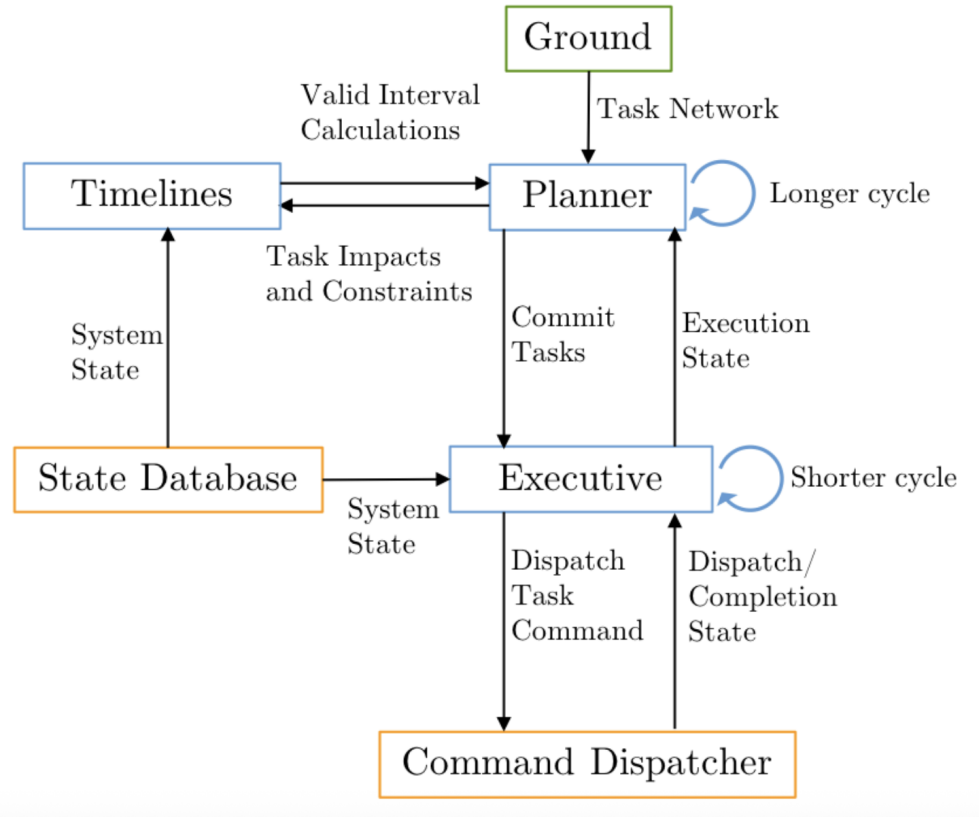


JPL's onboard planner and executive. Ground uplinks a **task network**: tasks, the **timelines** they read and write, and the constraints between them.

MEXEC schedules it onboard and repairs the plan as it executes.

The question: before we uplink a task network, can we know that it can be scheduled at all, and that every schedule it admits is safe?

The setting: MEXEC



JPL's onboard planner and executive. Ground uplinks a **task network**: tasks, the **timelines** they read and write, and the constraints between them.

MEXEC schedules it onboard and repairs the plan as it executes.

The question: before we uplink a task network, can we know that it can be scheduled at all, and that every schedule it admits is safe?

TaskSAT

A DSL for task networks, as close as we could get to MEXEC's concepts and semantics, plus temporal logic. Solved with Z3.

Question		**Form**
Validity	Is there any schedule? (the planning problem)	$\exists i \, \exists s \, . \, \mathit{valid}(i, s)$
Properties	Is every schedule safe?	$\forall i \, \forall s \, . \, \mathit{valid}(i, s) \Rightarrow \varphi$
Realizability	Is there a schedule from *every* initial state?	$\forall i \, \exists s \, . \, \mathit{valid}(i, s)$
Compositional	Does it hold for *any* number of repetitions?	$\{P\} \, S \, \{P\} \Rightarrow \forall N . \{P\} \, S^N \, \{P\}$

i is an initial state, s a schedule.

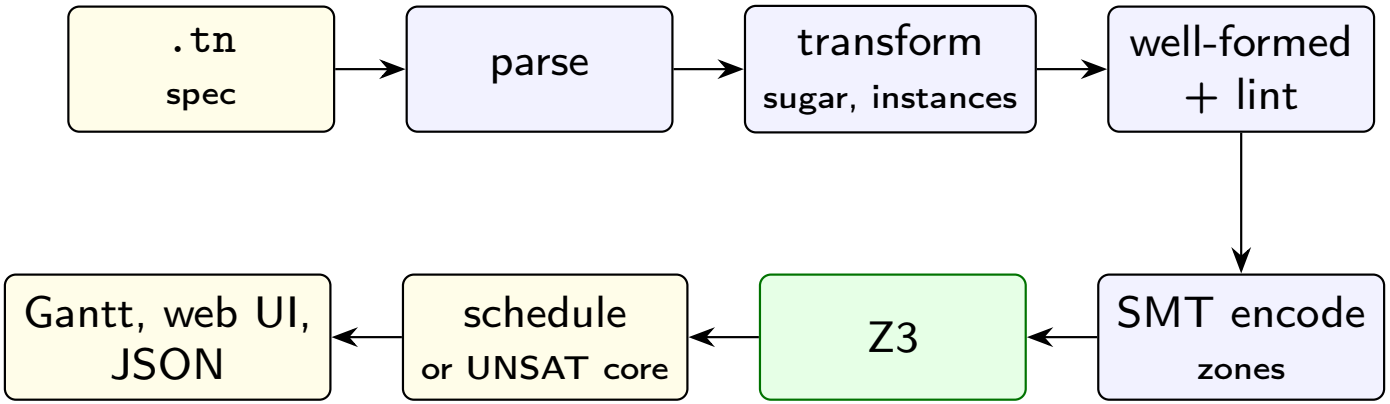
TaskSAT

A DSL for task networks, as close as we could get to MEXEC's concepts and semantics, plus temporal logic. Solved with Z3.

Question		**Form**
Validity	Is there any schedule? (the planning problem)	$\exists i \, \exists s \, . \; \mathit{valid}(i, s)$
Properties	Is every schedule safe?	$\forall i \, \forall s \, . \; \mathit{valid}(i, s) \Rightarrow \varphi$
Realizability	Is there a schedule from *every* initial state?	$\forall i \, \exists s \, . \; \mathit{valid}(i, s)$
Compositional	Does it hold for *any* number of repetitions?	$\{P\} \, S \, \{P\} \Rightarrow \forall N . \{P\} \, S^N \, \{P\}$

i is an initial state, s a schedule.

The pipeline



- Much of the language is **sugar**: task ranges, sessions, mutex, auto-instantiation are all flattened before encoding.
- The encoding is **quantifier-free**: one formula, one solver call per question.
- Python, PLY, Z3, Flask. Open source: `github.com/nasa-jpl/tasksat`

A rover: timelines

```
tasknet Rover2 {
  end = 400;

  timelines {
    arm         : atomic = 0;
    location    : state(home, target) = home;
    data        : cumulative [0.0, 50.0]  bounds [0.0, 100.0] = 0.0;
    battery     : rate       [10.0, 100.0] bounds [0.0, 100.0] = 60.0;
    temperature : rate       [5.0, 40.0]  bounds [0.0, 100.0] = 10.0;
  }
  ...
}
```

- **atomic**: a $[0, 1]$ claim, for mutual exclusion.
- **state**: an enumeration.
- **cumulative**: a number, changed in steps.
- **rate**: a number with a rate of change: a battery, a temperature.
- The first interval must hold at all times. The **bounds** clamp.

A rover: tasks

```
taskdef drive_def {
  pre {
    battery in [60.0, 100.0];
    temperature in [10.0, 40.0];
  }
  impacts {
    maint { battery -~ 1.5; }
    post  { location = target; }
  }
}

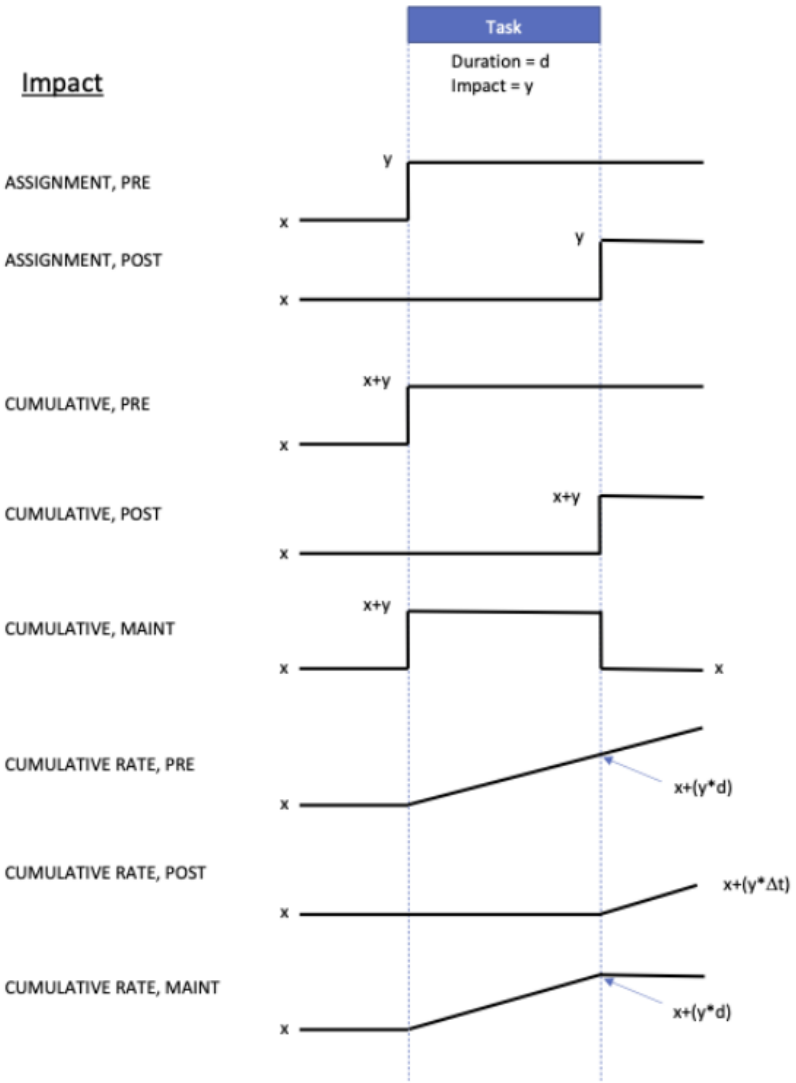
task drive : drive_def {
  start_range    [100, 120];
  end_range      [130, 160];
  duration_range [30, 40];
}
```

```
task collect {
  duration_range [20, 30];
  pre {
    location = target;
    battery in [60.0, 100.0];
    arm = 0;
  }
  impacts {
    maint { arm += 1;
            battery -~ 0.5; }
    post  { data += 40.0; }
  }
}

optional task charge : charge_def {
  duration_range [60, 70];
}
```

`-~` / `+~` change a **rate**; `+=` and `=` change a **value**. An **optional** task is scheduled only if it is needed.

When impacts happen



- **pre**: at the start, and stays.
- **post**: at the end, and stays.
- **maint**: at the start, undone at the end.

A task's own `pre` sees the state *before* its impacts: `collect` checks `arm = 0`, then runs with `arm = 1`.

These are MEXEC's semantics, as best we understand them.

Constraints and properties

```
constraints {
  prop not_charge_drive:   always not (active(charge) and active(drive));
  prop not_charge_collect: always not (active(charge) and active(collect));
  prop temperature10:      always temperature >= 10;
}
```

Constraints restrict which schedules are valid.

```
properties {
  prop target_reached: eventually (location = target and data >= 30);
  prop drive_charge:   always (active(drive) -> eventually active(charge));
  prop temperature:    always temperature >= 10;
}
```

Properties are proved for *all* valid schedules, or refuted with a counterexample.

Past-time operators too: `since`, `once`, `sofar`.

Constraints and properties

```
constraints {
  prop not_charge_drive:   always not (active(charge) and active(drive));
  prop not_charge_collect: always not (active(charge) and active(collect));
  prop temperature10:      always temperature >= 10;
}
```

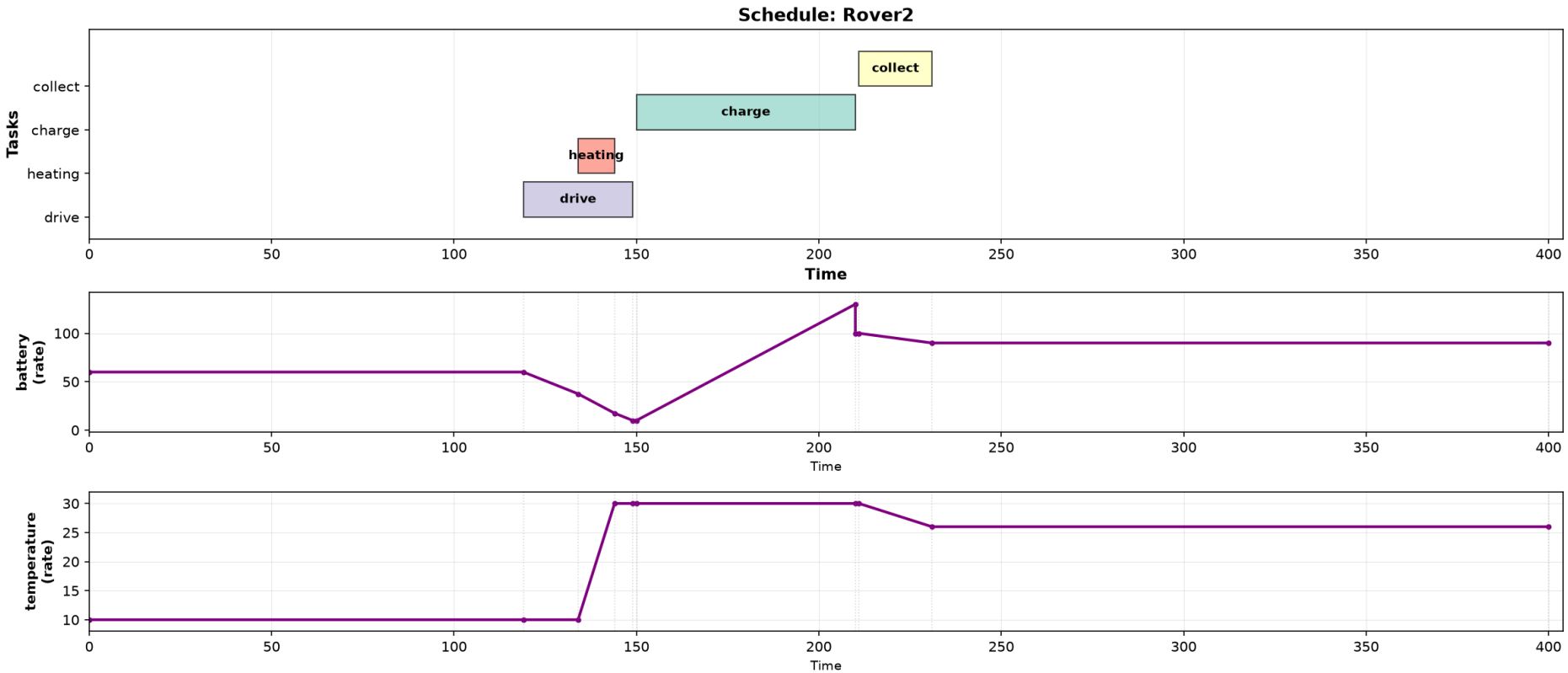
Constraints restrict which schedules are valid.

```
properties {
  prop target_reached: eventually (location = target and data >= 30);
  prop drive_charge:   always (active(drive) -> eventually active(charge));
  prop temperature:    always temperature >= 10;
}
```

Properties are proved for *all* valid schedules, or refuted with a counterexample.

Past-time operators too: `since`, `once`, `sofar`.

The schedule



Drive first, from the 60% we start with, down to 10%. Then charge to 100% and collect. `heating` keeps the temperature up. Charging first would waste most of it: the battery is full after 20 of its ≥ 60 time units.

The properties

Verification Checks json json — 4/4 passed

All checks that apply to this mode (non-compositional) are listed. The **Form** column shows the logical shape (∃ = exists, ∀ = for all; over initial state, then schedule). Rows marked not run or no properties had nothing to verify and do not count as failures.

Status	Check	Form	Formula / Detail	Time	Actions
✓	**Validity**	∃∃	`exists initial state i . exists schedule s . valid(i, s)` A valid schedule exists.	2.887s	
✓	**Properties**	∀∀	`forall initial state i . forall schedule s . valid(i, s) -> property` 3 temporal properties checked (each holds for every valid schedule).	2.975s	
✓	└ `target_reached`		`eventually (location = target and data >= 30.0)`	0.052s	
✓	└ `drive_charge`		`always (__drive_active = 1 -> eventually __charge_active = 1)`	2.88s	
✓	└ `temperature`		`always temperature >= 10.0`	0.041s	
⊖	Realizability	∀∃	Not run — enable with `--realizability`.		
⊖	Compositional Proof	∀∀·∀∃	Not run — enable with `--compositional` (or an `invariant compositional {...}` block).		
			Total verification time	**5.903s**	

Validity and the three properties, each with its logical form. 6 seconds in total.

When there is no schedule

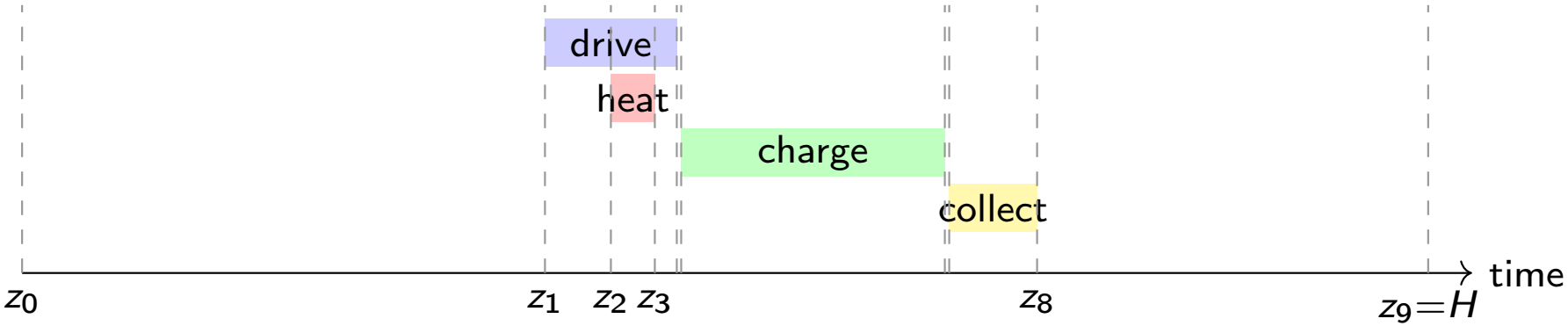
Ask for charge to come first:

```
task drive : drive_def {
   ...
   after charge;
}
```

```
=== Phase 1: Validity Check (EE) ===
UNSAT CORE (126 conflicting constraints)
  ...
  1. task_start_range: task 'drive' start must be in [100,120]
  2. task_end_range: task 'drive' end must be in [130,160]
  3. task_duration_range: task 'drive' duration must be in [30,40]
  4. dependency_after: 'drive' after 'charge'
  ...
  6. task_duration_range: task 'charge' duration must be in [60,70]
```

Charging first fills the battery (clamped at 100), drive takes it to at most 55, and collect needs 60. There is only one charge. Every constraint is labelled, so the core reads back as the spec.

Zones: time is cut at task boundaries



- Zone boundaries $z_0 < z_1 < \ldots$ are SMT variables; every task start and end **is** a zone boundary, and vice versa. Nothing changes inside a zone.
- Each timeline gets **one value per zone**, so the formula is $O(\#tasks \times \#timelines)$, independent of the horizon.
- Impacts are case splits on which boundary a task's start or end equals.

The one nonlinear term

A rate timeline ℓ has a value $v[j]$ and a rate $\rho[j]$ per zone:

$$v[j{+}1] \;=\; \mathit{clamp}\big(\, v[j] \;+\; \underbrace{\rho[j] \cdot (z_{j+1} - z_j)}_{\text{rate} \times \text{zone width}} \;+\; \Delta[j] \,\big)$$

Both factors are unknowns: the only product of two variables in the encoding.

- It is what makes batteries hard. A network where many tasks drain one battery couples them all through these products.
- It is also an opportunity: **fix the times, and the formula becomes linear.** Realizability exploits exactly this.

The one nonlinear term

A rate timeline ℓ has a value $v[j]$ and a rate $\rho[j]$ per zone:

$$v[j{+}1] \;=\; \mathit{clamp}\big(\, v[j] \;+\; \underbrace{\rho[j] \cdot (z_{j+1} - z_j)}_{\text{rate} \times \text{zone width}} \;+\; \Delta[j] \,\big)$$

Both factors are unknowns: the only product of two variables in the encoding.

- It is what makes batteries hard. A network where many tasks drain one battery couples them all through these products.
- It is also an opportunity: **fix the times, and the formula becomes linear**. Realizability exploits exactly this.

Checking a property

Temporal formulas are unfolded over the zones, e.g.

$$\texttt{always}\ \varphi \;=\; \bigwedge_j \varphi[j] \qquad \texttt{eventually}\ \varphi \;=\; \bigvee_j \varphi[j]$$

To prove φ for *all* schedules, ask Z3 for a schedule violating it:

$$\exists i\, \exists s\,.\; \mathit{valid}(i, s) \wedge \neg\varphi(s)$$

- UNSAT: φ **HOLDS**, for every schedule.
- SAT: **VIOLATED**, and the model is the counterexample schedule.

Same idea for `final { ... }`: the state at the end of every schedule.

Realizability: the vacuity trap

```
initial { battery in [0.0, 59.0]; }

task work {
  start_range [10, 10];  duration 20;
  pre      { battery in [30.0, 100.0]; }
  impacts { post { battery += 20.0; } }
}
```

- **Validity** passes: Z3 picks a battery ≥ 30.
- **Properties** are vacuously true for battery < 30: there are no schedules to violate them.

$\forall i \exists s$ finds the problem:

```
=== Phase 3: Realizability Check (AE) ===
  [iter 1] trying initial state: battery = 0
  -> VIOLATED!
  Initial state with no valid schedule:
    battery = 0
```

Realizability: the vacuity trap

```
initial { battery in [0.0, 59.0]; }

task work {
  start_range [10, 10];  duration 20;
  pre      { battery in [30.0, 100.0]; }
  impacts { post { battery += 20.0; } }
}
```

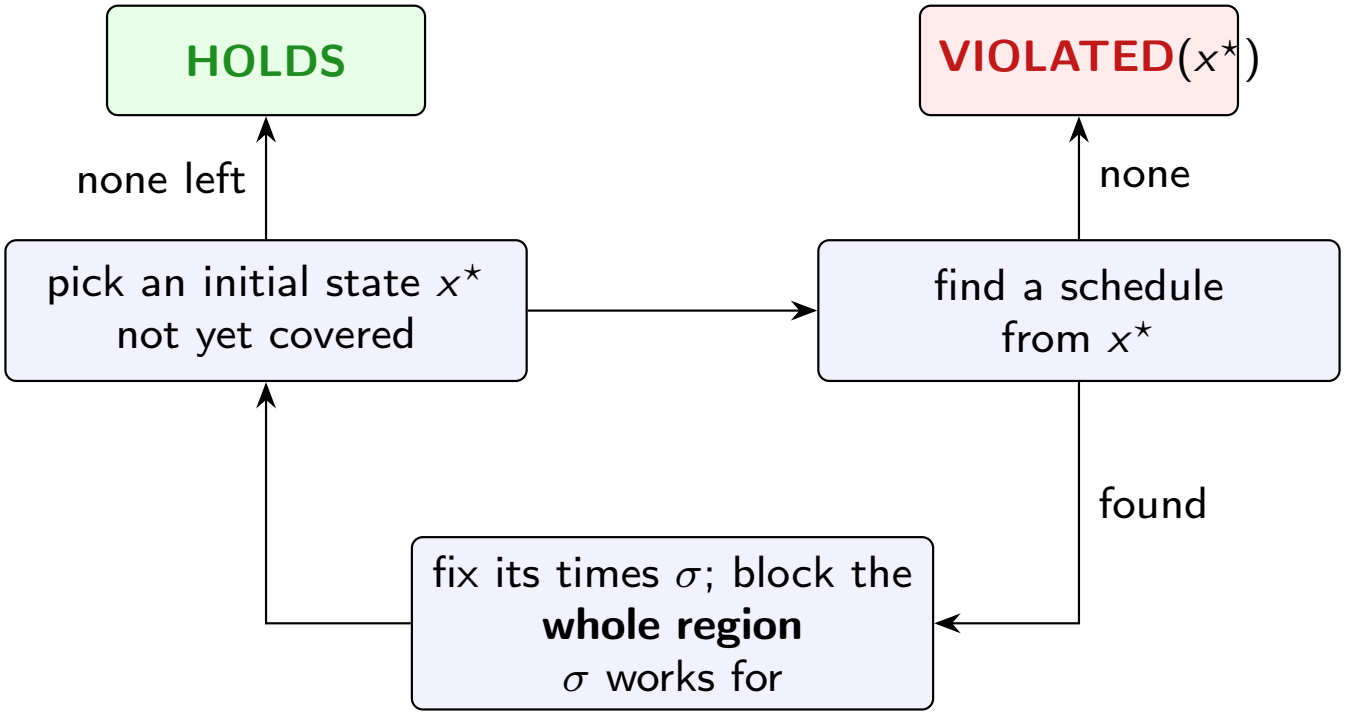
- **Validity** passes: Z3 picks a battery ≥ 30.
- **Properties** are vacuously true for battery < 30: there are no schedules to violate them.

$\forall i \, \exists s$ finds the problem:

```
=== Phase 3: Realizability Check (AE) ===
  [iter 1] trying initial state: battery = 0
  -> VIOLATED!
  Initial state with no valid schedule:
    battery = 0
```

Realizability by CEGIS

$\forall\exists$ over nonlinear arithmetic is too hard to hand to Z3 directly. Instead:



- With the times fixed, the region is a **linear** formula in x: exact, not sampled.
- One schedule typically covers a large region, so few iterations are needed.

Compositional: verify once, repeat for free

Missions repeat: drive, localize, downlink, … 200 times. The solve grows super-linearly with N.

```
taskdef Cycle {                          // a session
  task activate   : Activate;
  task deactivate : Deactivate { after activate; }
}
invariant compositional { mode = idle; }   // P

task cycle1  : Cycle;
task cycle2  : Cycle { after cycle1; }   ...
task cycle20 : Cycle { after cycle19; }
```

Project to **one** session S and prove $\{P\}\,S\,\{P\}$ with **two** checks:

- **Safety** ($\forall\forall$): every schedule from P ends in P.
- **Realizability** ($\forall\exists$): from every state in P, *some* schedule ends in P.

Safety alone is the vacuity trap again. Both together give $\forall N.\ \{P\}\,S^N\{P\}$.

```
=== Compositional Proof: {P}S{P} => forall N {P}S^N{P} ===
  -> HOLDS (safety=holds, realizability=holds)        -- 0.05 s, for any N
```

Compositional: verify once, repeat for free

Missions repeat: drive, localize, downlink, … 200 times. The solve grows super-linearly with N.

```
taskdef Cycle {                          // a session
  task activate   : Activate;
  task deactivate : Deactivate { after activate; }
}
invariant compositional { mode = idle; }   // P

task cycle1  : Cycle;
task cycle2  : Cycle { after cycle1; }   ...
task cycle20 : Cycle { after cycle19; }
```

Project to **one** session S and prove $\{P\}\,S\,\{P\}$ with **two** checks:

- **Safety** ($\forall\forall$): every schedule from P ends in P.
- **Realizability** ($\forall\exists$): from every state in P, *some* schedule ends in P.

Safety alone is the vacuity trap again. Both together give $\forall N.\ \{P\}\,S^N\{P\}$.

```
=== Compositional Proof: {P}S{P} => forall N {P}S^N{P} ===
  -> HOLDS (safety=holds, realizability=holds)        -- 0.05 s, for any N
```

Compositional: the fine print

- **Uniformity is enforced.** All session instances must be interchangeable; otherwise a P-preserving session followed by a P-breaking one would give a false **HOLDS**. Heterogeneous networks are refused.
- **No gap drift is assumed** between sessions (a battery draining while idle).

On a real MEXEC network (199 waypoints):

- The full solve does not finish. The cost is the single shared **battery rate timeline**, not symmetry and not the number of zones.
- As flown, there is no repeating unit: driving and downlink run on independent clocks.
- A faithful re-model with one repeating mission cycle **HOLDS** in 34 seconds, independent of N.

The method works once you **supply** a repeating unit. Finding one is modelling, not solving.

Compositional: the fine print

- **Uniformity is enforced.** All session instances must be interchangeable; otherwise a P-preserving session followed by a P-breaking one would give a false **HOLDS**. Heterogeneous networks are refused.
- **No gap drift is assumed** between sessions (a battery draining while idle).

On a real MEXEC network (199 waypoints):

- The full solve does not finish. The cost is the single shared **battery rate timeline**, not symmetry and not the number of zones.
- As flown, there is no repeating unit: driving and downlink run on independent clocks.
- A faithful re-model with one repeating mission cycle **HOLDS** in 34 seconds, independent of N.

The method works once you **supply** a repeating unit. Finding one is modelling, not solving.

Lint: cheap explanations before the solve

Static checks that run on every verification, in time linear in the spec. They never block a run: the solver stays the authority.

Rule	**Finds**
`unsatisfiable-initial`	nothing writes T, and its initial value fails some condition on T
`unsupported-transition`	a task's `pre` and `inv` on T admit disjoint values, and it does not change T
`unused-timeline`	declared, never read or written
`no-op-impact`	an impact of 0

A 50-downlink MEXEC network lints in 0.17 s; it does not solve in 240 s. The findings are *advisory*: a timeline nothing in the plan writes may be set from the ground.

Conclusion

1. A task network is a **logical formula**. Scheduling is $\exists\exists$; verification is $\forall\forall$.
2. The interesting questions are the **other quantifier orders**. $\forall\exists$ closes the vacuity trap that $\forall\forall$ leaves open.
3. The zone encoding is linear except for **rate** $\times$ **duration**. That one product sets the scaling limit, and fixing the times removes it.
4. Scaling to real missions needs **structure**: repeating sessions, invariants, and eventually assume-guarantee contracts per cycle.
5. The hard part is not the solver. It is knowing what the spec **means**: MEXEC's semantics, and which properties are worth stating.

Docs and source: `https://nasa-jpl.github.io/tasksat/`

Conclusion

1. A task network is a **logical formula**. Scheduling is $\exists\exists$; verification is $\forall\forall$.
2. The interesting questions are the **other quantifier orders**. $\forall\exists$ closes the vacuity trap that $\forall\forall$ leaves open.
3. The zone encoding is linear except for **rate** $\times$ **duration**. That one product sets the scaling limit, and fixing the times removes it.
4. Scaling to real missions needs **structure**: repeating sessions, invariants, and eventually assume-guarantee contracts per cycle.
5. The hard part is not the solver. It is knowing what the spec **means**: MEXEC's semantics, and which properties are worth stating.

Docs and source: `https://nasa-jpl.github.io/tasksat/`

Conclusion

1. A task network is a **logical formula**. Scheduling is $\exists\exists$; verification is $\forall\forall$.
2. The interesting questions are the **other quantifier orders**. $\forall\exists$ closes the vacuity trap that $\forall\forall$ leaves open.
3. The zone encoding is linear except for **rate** $\times$ **duration**. That one product sets the scaling limit, and fixing the times removes it.
4. Scaling to real missions needs **structure**: repeating sessions, invariants, and eventually assume-guarantee contracts per cycle.
5. The hard part is not the solver. It is knowing what the spec **means**: MEXEC's semantics, and which properties are worth stating.

Docs and source: `https://nasa-jpl.github.io/tasksat/`

Conclusion

1. A task network is a **logical formula**. Scheduling is $\exists\exists$; verification is $\forall\forall$.
2. The interesting questions are the **other quantifier orders**. $\forall\exists$ closes the vacuity trap that $\forall\forall$ leaves open.
3. The zone encoding is linear except for **rate** $\times$ **duration**. That one product sets the scaling limit, and fixing the times removes it.
4. Scaling to real missions needs **structure**: repeating sessions, invariants, and eventually assume-guarantee contracts per cycle.
5. The hard part is not the solver. It is knowing what the spec **means**: MEXEC's semantics, and which properties are worth stating.

Docs and source: `https://nasa-jpl.github.io/tasksat/`

Conclusion

1. A task network is a **logical formula**. Scheduling is $\exists\exists$; verification is $\forall\forall$.
2. The interesting questions are the **other quantifier orders**. $\forall\exists$ closes the vacuity trap that $\forall\forall$ leaves open.
3. The zone encoding is linear except for **rate** $\times$ **duration**. That one product sets the scaling limit, and fixing the times removes it.
4. Scaling to real missions needs **structure**: repeating sessions, invariants, and eventually assume-guarantee contracts per cycle.
5. The hard part is not the solver. It is knowing what the spec **means**: MEXEC's semantics, and which properties are worth stating.

Docs and source: `https://nasa-jpl.github.io/tasksat/`

Questions?